Final project report for MORML

Donnarumma Luigi s343583
Iannarelli Stefano s345141

June 2025

Abstract

This work presents a comparison between three Model Order Reduction techniques: Proper Orthogonal Decomposition (POD), Physics-Informed Neural Networks (PINNs), and a hybrid approach (POD-NN). The methods are described and applied to a nonlinear partial differential equation (PDE), with results compared against a high-fidelity reference solution in terms of accuracy and computational efficiency. Two benchmark problems with different forcing terms are considered to assess the performance of the approaches.

1 Introduction

In many scientific and engineering applications, solving PDEs[1] is a central task. However, high-fidelity numerical simulations often come with significant computational costs. Model Order Reduction (MOR) techniques aim to reduce the complexity of such problems by projecting them onto low-dimensional subspaces, thus enabling faster computations while preserving acceptable accuracy. Among the most widely used MOR techniques is the Proper Orthogonal Decomposition (POD), which constructs a reduced basis by extracting dominant modes from a set of high-fidelity solutions (snapshots). More recently, Machine Learning (ML) has emerged as a powerful tool for learning complex mappings from data. Physics-Informed Neural Networks (PINNs) incorporate physical laws into the learning process by embedding the governing PDEs into the loss function, allowing for data-driven modeling with physical consistency. Hybrid approaches such as POD-NN combine the interpretability of reduced-order modeling with the flexibility of neural networks.

In this project, we analyze and compare the performance of these three approaches (POD, PINN, and POD-NN) on two different problems involving non linear parametric PDEs with parametric and non-parametric forcing terms, without affine property. Our goal is to evaluate the trade-off between accuracy and computational efficiency, and to explore the potential of data-driven methods in the context of MOR.

2 Explanation of the methods

In this section, we briefly describe the main methods employed in the project: Proper Orthogonal Decomposition (POD), Physics-Informed Neural Networks (PINNs), and POD combined with Neural Networks (POD-NN).[3]

We refer to an high-fidelity (HF) solution better presented in appendix [A]. It is also verified the convergence[2] of each order (one and two). To simplify, only the order one of the HF solution will be used to present model order reduction methods. The approach for higher orders of HF solutions remains the same.

2.1 Proper Orthogonal Decomposition (POD)

Proper Orthogonal Decomposition is a classical technique for model order reduction based on linear projection. Given a set of high-fidelity solution snapshots, POD performs a Singular Value Decomposition (SVD) to extract the most energetic modes. These modes form an orthonormal reduced basis that captures the dominant dynamics of the system (see more in appendix [B]).

The original problem is then projected onto this reduced basis, resulting in a low-dimensional system that is cheaper to solve. However, the accuracy of the reduced model depends on how well the reduced space can represent the solution manifold.

2.2 Physics-Informed Neural Networks (PINNs)

Physics-Informed Neural Networks are a class of machine learning models that approximate the solution of a PDE by minimizing a loss function that includes the residual of the governing equations. The main idea is to train a neural network to satisfy the PDE, the boundary conditions, and possibly some initial conditions, using automatic differentiation to compute the derivatives. Variational formulation is also possible, instead of computing derivatives, but this variant will be left out of our comparison.

PINNs do not rely on any predefined mesh or reduced basis and are particularly suitable for problems where data is scarce or noisy. However, their training can be computationally demanding and sensitive to the choice of hyperparameters.

Given a generic PDE

$$\begin{cases} \mathcal{A}(u) = f & \text{on} \quad \Omega \subseteq \mathbb{R}^d \\ \mathcal{B}(u) = g & \text{on} \quad \Gamma \subseteq \partial\Omega \end{cases}$$

the PINN operator is defined as

$$\begin{aligned} \mathcal{F}_{NN} &: \Omega \to \mathcal{F} \\ \mathbf{x} = (x, y; \boldsymbol{\mu}) &\mapsto u(\mathbf{x}; \boldsymbol{\mu}) \end{aligned}$$

where $\mathcal{F}$ is the space where the solution u lives (e.g. $C^0(\Omega)$ for the strong formulation or $H^1(\Omega)$ for the weak formulation). The network is trained with the following loss function (λ is an hyperparameter).

$$\begin{aligned} \mathcal{L}^*(v; \mathbf{x}) &= \mathcal{L}_{\mathcal{A}}(v; \mathbf{x}) + \lambda \mathcal{L}_{\mathcal{B}}(v; \mathbf{x}) \\ \mathcal{L}_{\mathcal{A}}(v; \mathbf{x}) &= \frac{1}{2} \left| f|_{\mathbf{x}} - \mathcal{A}(v)|_{\mathbf{x}} \right|^2 \\ \mathcal{L}_{\mathcal{B}}(v; \mathbf{x}) &= \frac{1}{2} \left| g|_{\mathbf{x}} - \mathcal{B}(v)|_{\mathbf{x}} \right|^2 \end{aligned}$$

2.3 POD-NN

The POD-NN method combines the advantages of projection-based model reduction and data-driven approximation. First, a reduced basis is computed using POD. Then, instead of solving the reduced-order system directly, a neural network is trained to learn the mapping from the parameter space to the reduced coordinates.

This two-stage approach leverages the efficiency of POD to reduce the dimensionality and uses a neural network to approximate the solution efficiently for new parameter values. It can achieve good generalization with lower computational cost compared to solving the full-order or reduced-order models directly.

Similarly to PINN we have an operator which refers to the Neural Network.

$$\begin{aligned} \Pi_{NN} &: \mathcal{P} \to V_N \\ \boldsymbol{\mu} &\mapsto u_N(\boldsymbol{\mu}) \end{aligned}$$

with $\mathcal{P}$ parametric space and V_N the reduced space. This time we have a supervised learning setting using a new loss

$$\mathcal{L}(\boldsymbol{\mu}) = \frac{1}{\#\text{ of Data}} \sum_{\boldsymbol{\mu}} \|\Pi_{NN}(\boldsymbol{\mu}) - u_\delta(\boldsymbol{\mu})\|$$

where using u_δ snapshot solution of the HF problem, $\mathbb{B}$ basis function of the POD and $\mathbf{A}$ matrix associated to the algebraic system of the HF problem, we can easily reconstruct u_N with the help of the projector $\mathbb{P} = (\mathbb{B}^T\mathbf{A}\mathbb{B})^{-1}\mathbb{B}^T\mathbf{A}$ because $u_N = \mathbb{P}u_\delta$.

2.4 Performance metrics definition

Before starting with the experiments and the presentation of the results, we would like to clarify the types of metrics that will be used from this point onward to compare the performance of the proposed methods. In particular, we evaluate both the accuracy and the computational efficiency of the reduced models with respect to the full-order model. We compute two standard error indicators: the squared absolute error and the squared relative error. These are based on a discrete inner product defined through a positive-definite matrix. Given the predicted solution u and the reference full-order solution u_{FOM}, the squared absolute error is computed as the weighted squared norm of their difference, while the squared relative error normalizes this quantity by the squared norm of the full-order solution itself. The formulas used are:

$$e^2_{\text{abs}} = (u_{\text{FOM}} - u)^T\mathbf{A}(u_{\text{FOM}} - u), \quad e^2_{\text{rel}} = \frac{e^2_{\text{abs}}}{u^T_{\text{FOM}}\mathbf{A}u_{\text{FOM}}}.$$

As for computational performance, we report the speed-up factor obtained by each method, defined as the ratio between the time required by the full-order model and that of the reduced model:

$$\text{Speed-Up} = \frac{t_{\text{FOM}}}{t_{\text{ROM}}}.$$

These quantities are evaluated on a test set of parameter instances and then averaged to obtain a global indication of model performance.

3 Experiment 1

We consider the two-dimensional spatial domain $\Omega = (0,1)^2$ and the parametric nonlinear elliptic PDE:

$$-\Delta u(\boldsymbol{\mu}) + \frac{\mu_0}{\mu_1}\left(e^{\mu_1 u(\boldsymbol{\mu})} - 1\right) = g(\boldsymbol{x};\boldsymbol{\mu}), \quad \text{for } \boldsymbol{x} \in \Omega, \tag{1}$$

with homogeneous Dirichlet boundary conditions on $\partial\Omega$, and where the parameter vector is $\boldsymbol{\mu} = (\mu_0, \mu_1) \in \mathcal{P} = [0.1, 1]^2$.

In this first case, the source term is independent of the parameters and is given by:

$$g(\boldsymbol{x};\boldsymbol{\mu}) = g_1(\boldsymbol{x}) = 100\sin(2\pi x_0)\cos(2\pi x_1), \quad \forall \boldsymbol{x} = (x_0, x_1) \in \Omega. \tag{2}$$

The tasks required for this problem are:

1. Solve the problem using the POD-Galerkin method over a Finite Element full-order model.
2. Solve the problem with a parametric Physics-Informed Neural Network (PINN).
3. Compare the two approaches in terms of computational cost and accuracy with respect to the full-order model.
4. Solve the problem using the POD-NN approach and compare it to the other two strategies.

3.1 POD

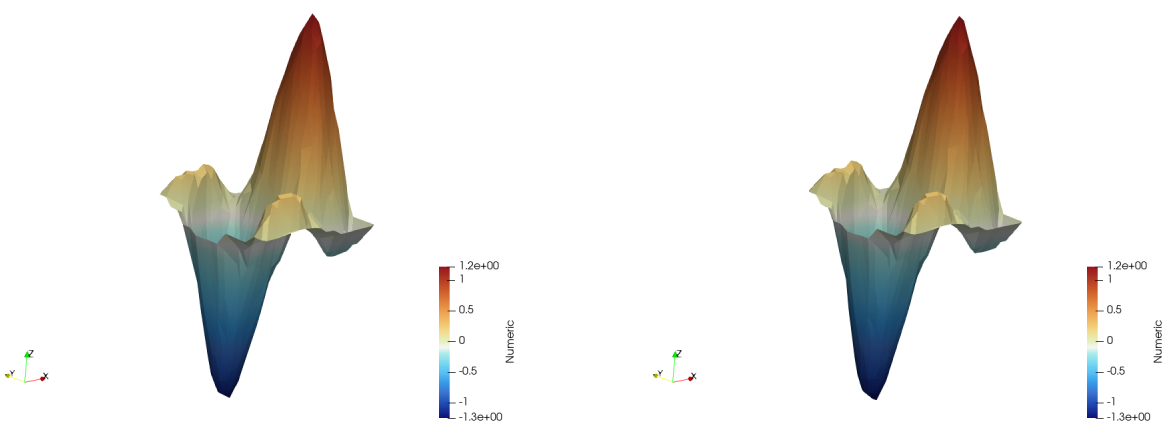
Before constructing a Reduced Order Model (ROM) using Proper Orthogonal Decomposition (POD), we firstly consider the fact we have a nonlinear problem to solve via Newton's method. We analytically computed the system to solve at the n-th Newton iteration with respect to δu, which is the following:

$$\int_\Omega \nabla\delta u \cdot \nabla v + \mu_0 e^{\mu_1 u_n}\,\delta u\, v = \int_\Omega -\nabla u_n \cdot \nabla v - \frac{\mu_0}{\mu_1}\left(e^{\mu_1 u_n} - 1\right) v + gv,$$

At each step we want to solve the equation with respect to δu in the reduced space by using POD. This decrease the computational cost while maintaining good accuracy.

The procedure we adopted, and so the overall structure of the implemented code, is mainly divided into an offline phase and an online phase:

- Offline phase
  - Generate a set of high-fidelity (FOM) solutions $u^{(i)}$ for different parameter values μ using finite element discretization, assembling 300 snapshots.
  - Compute the POD basis by performing Singular Value Decomposition (SVD) of the snapshot matrix using the H^1 scalar product, selecting the number of modes N such that the retained energy tolerance is below 10^{-7}, resulting in $N = 3$.
  - Preassemble all matrices and terms that do not depend on the solution during the offline phase.
- Online phase:
  - In the online phase, solve the reduced nonlinear system by projecting the full PDE onto the reduced space and applying Newton's method, reassembling at each iteration only the matrices depending on the current solution.
  - Compute comparison metrics with respect to the FOM reference, including absolute and relative errors and speed-up.



(a) High-fidelity solution.

(b) POD solution.

Figure 1: Comparison between the high-fidelity (HF) and reduced-order (ROM) solutions for test parameters $\mu_0 = 0.455$ and $\mu_1 = 0.72$.

Table 1: Performance metrics for POD (Experiment 1)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
POD	0.0072	0.0017	1.53

Although a model reduction technique such as POD often yields speed-ups of one or two orders of magnitude in linear problems, the nonlinearity of our PDE significantly limits the performance. In fact, due to the presence of a nonlinear term, the reduced-order model must still be solved iteratively using Newton's method, requiring the reassembly and projection of some matrices of the Jacobian and residual at each step. In particular, only the stiffness matrix and the forcing term are preassembled in the offline phase. This leads to a more moderate speed-up of approximately $2\times$.

3.2 PINN

The key idea behind PINNs is to approximate the solution of a differential equation by training a neural network that satisfies the governing equations and boundary conditions in a weak sense.

The neural network receives the spatial coordinates and the parameter $\mu = (\mu_0, \mu_1)$ as input and outputs an approximation of the solution $u(x, y; \mu_0, \mu_1)$. The loss function is composed of two terms: a residual loss enforcing the PDE equation at collocation points inside the domain; and a boundary loss enforcing Dirichlet boundary conditions.

$$\mathcal{L} = \mathcal{L}_{\text{PDE}} + \lambda \, \mathcal{L}_{\text{BC}}$$

λ here is an Hyperparameter used to perform efficient training also on the boundary points because $\mathcal{L}_{\text{BC}}$ could be of many orders of magnitude different from $\mathcal{L}_{\text{PDE}}$, leading to an unfair optimization of one of the two conditions.

The offline and online phase here are divided as follows:

- Offline phase
  - Definition of the net and training with random collocation points
- Online phase:
  - Given the mesh coordinates and new parameters, evaluate the neural network to obtain the coefficients of the solution already in the higher space.
  - Compute comparison metrics with respect to the FOM reference

Before showing the results, a brief explanation of the neural network we used is essential because of its importance in this method. It is a fully connected feed-forward neural network with 3 residual blocks (each of them made of three linear layer) and other two linear hidden layers. They all have 128 neurons, but the first layer takes 4 tensors as input (the x,y coordinates and the corresponding μ_0, μ_1) while the last layer gives in output just one tensor with the solution coefficients for the mesh points.
The training here is the crucial part. We trained using the Adam optimizer combined with the Cosine Annealing scheduler. We experimented with several combinations of hyperparameters to identify the most suitable configuration for our problem. However, due to limited computational resources, we were not able to explore as many combinations as we would have liked. The set of hyperparameters selected based on our constrained search is reported in table 2.

Epochs	**Learning Rate**	$\boldsymbol{\lambda}$	**Scheduler** T_{max}	**Scheduler** η_{min}
10000	1×10^{-4}	1×10^{4}	100	1×10^{-6}

Table 2: Selected hyperparameters after a limited search.

The behavior of the training loss with the selected hyperparameter is almost as expected and it is shown in figure 2a. It oscillates more with increasing the epochs but we considered the overall trend acceptable. Note that the initial drop in the MSE BC could be due to the reduction of the boundary condition loss. At the beginning, this term is more relevant because of the λ factor. After some epochs, the system likely starts to realize that it also needs to reduce the PDE loss, especially as the two loss terms become of the same order of magnitude.

Table 3: Performance metrics for PINN (Experiment 1)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
PINN	0.296	0.0694	27.4

The results are shown in table 3. The errors are pretty high, reaching an average relative error of almost the 0.0694 which for many application could be not admissible, indeed the predicted solution is precise only

at the first decimal digit. On the other hand, the Speed-up is pretty good. We also have to consider the pros of this method as the advantage of mesh-free implementation and flexibility with respect to complex geometries that now are not the cases of our problem. Moreover, the solution has some very intense gradients and this affects PINN's performances in general. Probably, with a better exploration of the hyperparameters used in the training phase, better results could be obtained.

Following the unsatisfactory results of the initial experiment, we decided to simplify the neural network architecture and adjust the training strategy. Specifically, we reduced the network to a much lighter structure composed of only 5 fully connected linear layers, each with 20 neurons. This choice was motivated by the need to decrease training time and reduce the complexity of the optimization problem, especially considering the computational resources available.

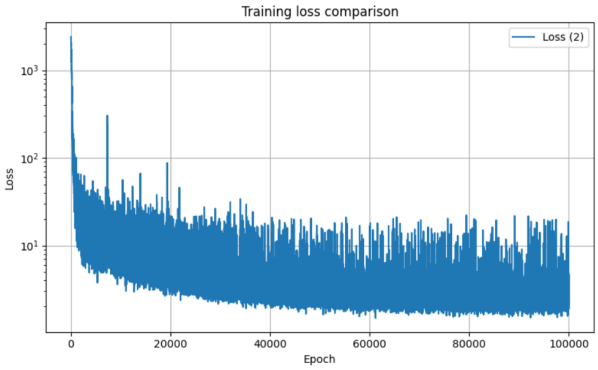
Additionally, instead of including the Dirichlet boundary conditions as a term in the loss function, we chose to enforce them directly within the forward pass of the neural network. This way, the network is constrained to produce outputs that inherently satisfy the boundary conditions, removing the need to balance the PDE and BC losses through a hyperparameter like λ. This approach avoids potential issues related to the relative scale of the loss components and simplifies the overall training process (figure 2b).

The new setup resulted in almost the same accuracy compared to the previous experiment, as shown in Table 4. The real impressive progress is the Speed-up, which is of an order of magnitude bigger than before.

Table 4: Performance metrics for PINN (Experiment 1 - Simplified architecture)

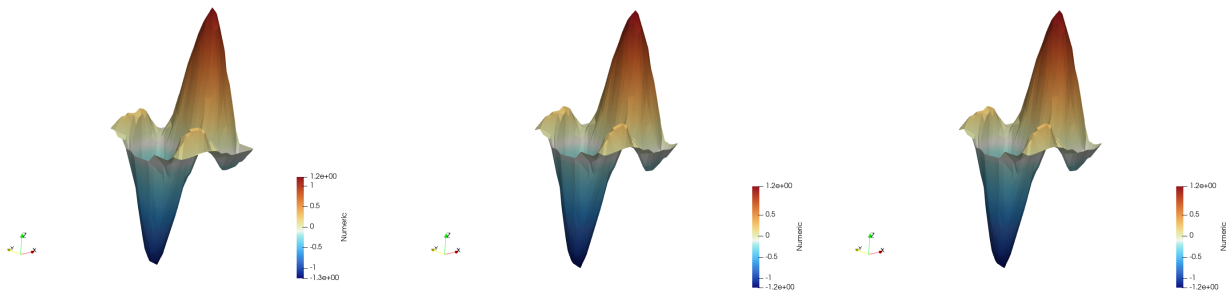
Method	Avg Abs Error	Avg Rel Error	Speed-Up
PINN (Simplified)	0.300	0.070	326.9

(a) Total, PDE and BC loss.



(b) Residual at final iteration.

Figure 2: On the left: Total training loss (Blue), PDE loss (Green), and BC loss (Orange) of the PINN network for Experiment 1. On the right: Training loss of the simplified architecture with imposed boundary conditions.



(a) High-fidelity solution. (b) PINN prediction. (c) Simplified PINN prediction.

Figure 3: Comparison between the high-fidelity solution, the PINN approximation, and the PINN with a simplified architecture. $\mu_0 = 0.455$, $\mu_1 = 0.72$.

This improvement highlights the importance of architectural choices and training strategies in PINN performance. In particular, enforcing boundary conditions explicitly and using a simpler network helped the model focus more efficiently on minimizing the PDE residuals, leading to a significant speed-up without affecting the solution accuracy compared to the more complex network. While further enhancements are still possible, especially with more systematic hyperparameter tuning, these results demonstrate the flexibility of the PINN approach even in low-resource scenarios.

3.3 PODnn

The POD-NN method enhances the standard POD framework by replacing the online Newton iterations with a neural network approximation. Once a reduced basis is extracted using POD, we train a neural network to learn the mapping between the input parameters (μ_0,μ_1) and the corresponding reduced coefficients.

This results in a faster evaluation of the solution, eliminating the need for reassembling and solving the system at runtime.

- **Offline phase**:
  - Solve the high-fidelity (FOM) problem for 300 different parameter pairs (μ_0, μ_1), to create the snapshots and the basis matrix as in the POD case. Indeed, the same reduced basis with $N = 3$ modes is reused here.
  - Train a fully-connected neural network to approximate the mapping

    $$\boldsymbol{\mu} = (\mu_0, \mu_1) \mapsto u_N(\boldsymbol{\mu}),$$

    using the reduced dataset as input-output pairs.
- **Online phase**:
  - Given new input parameters, evaluate the neural network to obtain the predicted reduced coefficients.
  - Reconstruct the full solution by projecting the reduced one.
  - Compare with the corresponding FOM solution to evaluate performance metrics.

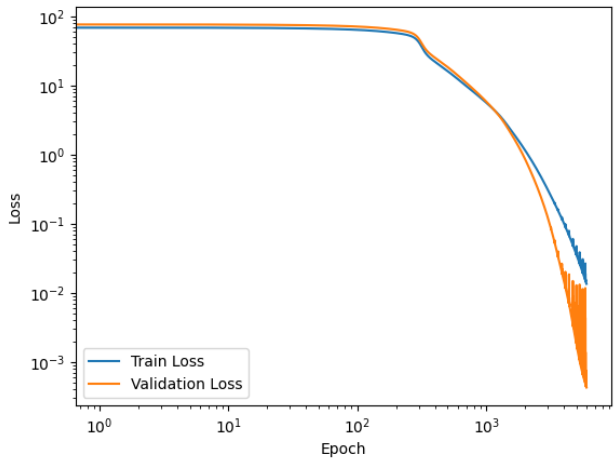
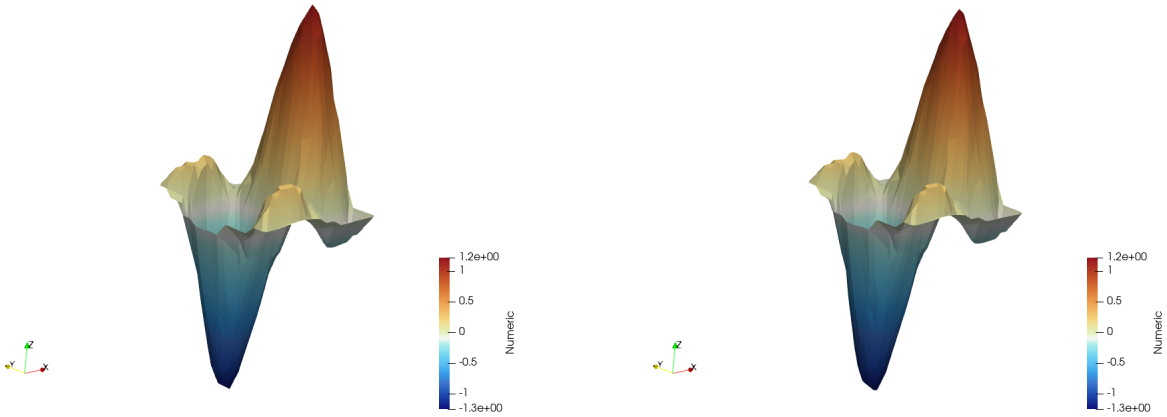


Figure 4: Training loss (Blue) and validation loss (Orange) of the PODnn network for Ex.1.



(a) High-fidelity solution.

(b) POD-NN solution.

Figure 5: Comparison between high-fidelity and POD-NN solutions for test parameters $\mu_0 = 0.455$ and $\mu_1 = 0.72$.

Table 5: Performance metrics for POD-NN (Experiment 1)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
POD-NN	0.0095	0.0022	35.4

The accuracy of the POD-NN approach remains comparable to the reduced model solved with Newton's method, while the computational cost is drastically reduced. This enables rapid evaluations, making POD-NN particularly suited for scenarios requiring real-time performance or intensive parameter sweeps. As shown in Figure 4, training could have been extended further; however, based on previous experiments with different hyperparameters, significant oscillations typically indicate that further improvements would be marginal compared to the additional computational cost. Training was thus stopped, as the attained accuracy was considered sufficient.

3.4 Comparison between methods for ex.1

Table 6 summarizes the accuracy and computational efficiency of the different methods tested in Experiment 1. The High-Fidelity (HF) model serves as the benchmark with perfect accuracy but the highest computational cost. POD delivers excellent accuracy with negligible error, while providing a poor speed-up over HF. Introducing the neural network surrogate in POD-NN leads to a slight increase in error but offers a substantial gain in evaluation speed. The PINN approach, both in its original and simplified forms, exhibits significantly larger errors compared to POD-based methods. However, the simplified PINN architecture achieves an extraordinary speed-up, over an order of magnitude higher than the original PINN, although without any improvement in accuracy. This comparison highlights the inherent trade-offs: POD-based methods prioritize accuracy with moderate speed-ups, while PINNs favor computational efficiency at the expense of precision. The simplified PINN, by embedding boundary conditions directly, shows how architectural adjustments can drastically accelerate predictions but still struggle to match the accuracy of classical reduced-order models. The best trade-off could actually be the POD-NN approach if combined with architectural simplifications similar to those applied to the PINN. Unfortunately, due to time constraints, we were not able to experiment with a simplified POD-NN architecture and thus could not evaluate the potential additional speed-up it might offer.

Table 6: Performance comparison of methods for Experiment 1

Method	Avg Abs Error	Avg Rel Error	Speed-Up
HF	0	0	1
POD	0.0072	0.0017	1.53
PINN	0.2960	0.0694	27.4
PINN (Simplified)	0.300	0.070	326.9
POD-NN	0.0095	0.0022	35.4

4 Experiment 2

In this second case, the source term depends on the parameter μ_0 and is defined as:

$$g(\boldsymbol{x};\boldsymbol{\mu}) = g_2(\boldsymbol{x};\mu_0) = 100\sin(2\pi\mu_0 x_0)\cos(2\pi\mu_0 x_1), \quad \forall \boldsymbol{x} = (x_0, x_1) \in \Omega. \tag{3}$$

The tasks are analogous to those in Experiment 1:

1. Solve the problem using the POD-Galerkin method over a Finite Element full-order model.
2. Solve the problem with a parametric PINN, using the same neural network architecture (number of layers and neurons per layer) as in Part 1.
3. Compare the two approaches in terms of computational cost and accuracy with respect to the full-order model.
4. Solve the problem using the POD-NN approach and compare it to the other two strategies.

4.1 POD

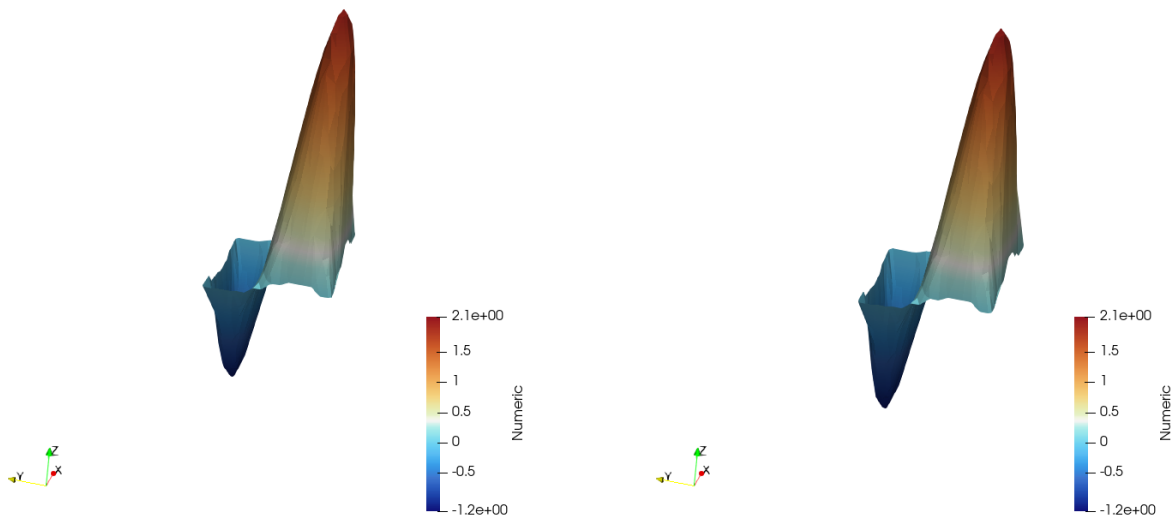
As in the previous experiment, the nonlinear PDE is solved using a Newton-Raphson scheme. The structure of the Newton iteration is the same as described in Experiment 1, with the only difference being the expression of the forcing term g, which in this case is parameter-dependent. This introduces an additional variability in the solution manifold.

To construct a Reduced Order Model (ROM) using Proper Orthogonal Decomposition (POD), we adopted a two-stage procedure as before:

- **Offline phase**:

- Generate a set of high-fidelity (FOM) solutions $u^{(i)}$ for various values of the parameters μ_0 and μ_1, incorporating the variability in g, and assemble 300 snapshots.
- Compute the POD basis by performing Singular Value Decomposition (SVD) of the snapshot matrix using the $\mathbb{L}^2$ scalar product. We retain $N = 9$ modes, corresponding to a prescribed energy tolerance of 10^{-7}.
- Preassemble only the stiffness matrix because do not depend on the solution u.

- **Online phase**:
  - Project the nonlinear problem onto the reduced space and solve it using Newton's method, reassembling at each iteration the nonlinear matrices depending on the current solution.
  - Evaluate performance metrics with respect to the FOM, including absolute and relative errors, and the speed-up obtained by model reduction.



(a) High-fidelity solution.

(b) POD solution.

Figure 6: Comparison between high-fidelity and reduced-order solutions for test parameters $\mu_0 = 0.455$ and $\mu_1 = 0.72$.

Table 7: Performance metrics for POD (Experiment 2)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
POD	0.0094	0.0029	1.40

Although the presence of a parametric forcing term increases the dimensionality of the solution manifold, the ROM still manages to achieve accurate results with a modest number of modes. As in the previous case, the need to reassemble and project nonlinear terms at each Newton step limits the overall speed-up. However, the cost reduction remains significant and validates the use of POD in this nonlinear, parameter-dependent setting.

4.2 PINN

For the second test case, we adopted the same neural network architecture and training strategy previously described for Experiment 1, with the only difference being the use of a different source term $g(x, y)$. As such, no structural change to the network or loss formulation was required.

We retained the same hyperparameters, listed in Table 2, as the ones selected for Experiment 1, due to their satisfactory convergence behavior and the limited computational resources available. However, we note that this choice may not be fully optimal for the new problem.

The training loss evolution is reported in Figure 7a. As observed, the training again follows a reasonably stable descent, although the oscillations near convergence are more pronounced, likely due to the increased complexity of the target solution.

Table 8: Performance metrics for PINN (Experiment 2)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
PINN	0.519	0.0877	29.5

As shown in Table 8, the PINN method for Experiment 2 results in slightly worse performance in terms of accuracy compared to Experiment 1. The average relative error reaches over 8×10^{-2}, which may be unacceptable in high-precision scenarios. Nonetheless, the method still achieves a significant speed-up of approximately 29×.

This decline in accuracy is most likely due to the more complex behavior of the solution in Experiment 2, which includes steeper gradients and stronger nonlinearities. As with Experiment 1, improved results could potentially be achieved through a more targeted search for optimal training configurations or by employing more advanced architectures or adaptive weighting strategies within the loss function.

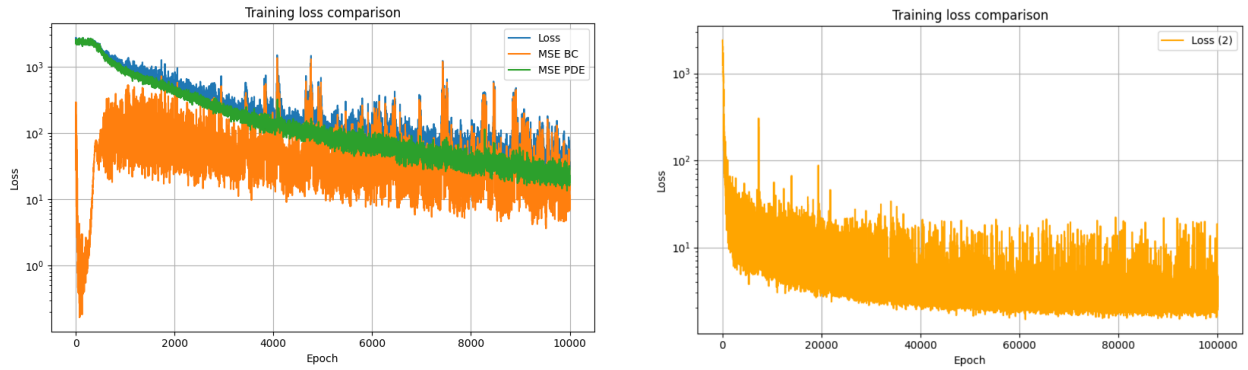
Given the unsatisfactory accuracy and Speed-up achieved in this second test case as well, we applied the same simplification strategy, and so the same simplified network, adopted in Experiment 1 to improve the results.

We enforced the Dirichlet boundary conditions directly in the forward pass of the neural network, rather than relying on an explicit loss term as well. This, again, ensure that the predicted solution always satisfies the prescribed boundary values by construction.

These changes significantly reduced training time, making possible the training for ten times the number of epochs used before. The new model produced a more stable training behavior (figure 7b) and a lower prediction error, as summarized in Table 9. The relative error dropped a bit but the main improvement is in the very significant speed-up reached.

Table 9: Performance metrics for PINN (Experiment 2 - Simplified architecture)

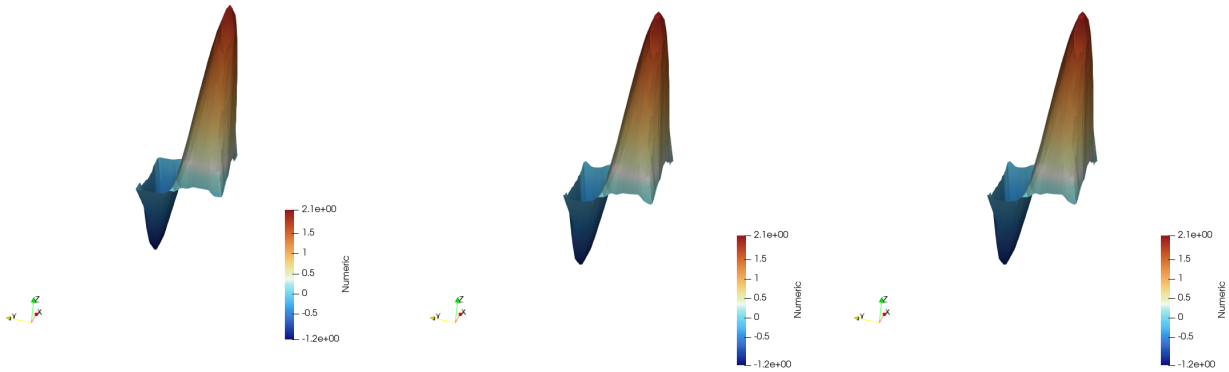
Method	Avg Abs Error	Avg Rel Error	Speed-Up
PINN (Simplified)	0.369	0.062	415.0



(a) Total, PDE and BC loss.

(b) Residual at final iteration.

Figure 7: On the left: Total training loss (Blue), PDE loss (Green), and BC loss (Orange) of the PINN network for Experiment 2. On the right: Training loss of the simplified architecture with imposed boundary conditions.



(a) High-fidelity solution.

(b) PINN prediction.

(c) Simplified PINN prediction.

Figure 8: Comparison between the high-fidelity solution, the PINN approximation, and the PINN with a simplified architecture. $\mu_0 = 0.455$, $\mu_1 = 0.72$.

These findings confirm the beneficial impact of architecture simplification and boundary condition enforcement within the network design. Despite the reduced model capacity, the PINN was able to capture the solution behavior more effectively, suggesting that a leaner network, when appropriately constrained, can outperform a deeper model.

4.3 PODnn

As explained before, we reused the first part of the offline phase of POD but than, instead of solving the reduced system iteratively, we use a Neural network to get the reduced coefficients. The structure of the net is the same as the experiment 1 in order to make a fair comparison of the differences between the parametric and the standard forcing term.

The implementation follows the same offline/online paradigm:

- **Offline phase**:
  - Generate high-fidelity solutions for 300 different parameter pairs (μ_0, μ_1), to create the snapshots and the basis matrix. Again, we reused the POD results getting the same basis matrix with $N = 9$.

– Train a fully-connected feedforward neural network (same as Ex.1) to approximate the map

$$\boldsymbol{\mu} = (\mu_0, \mu_1) \mapsto u_N(\boldsymbol{\mu}),$$

using a dataset of parameter-reduced solution pairs.

- **Online phase**:
  – For a new parameter (μ_0, μ_1), evaluate the neural network to obtain the reduced coefficients.
  – Reconstruct the approximate solution in the high-dimensional space reprojecting it.
  – Compare the output with the corresponding FOM solution.

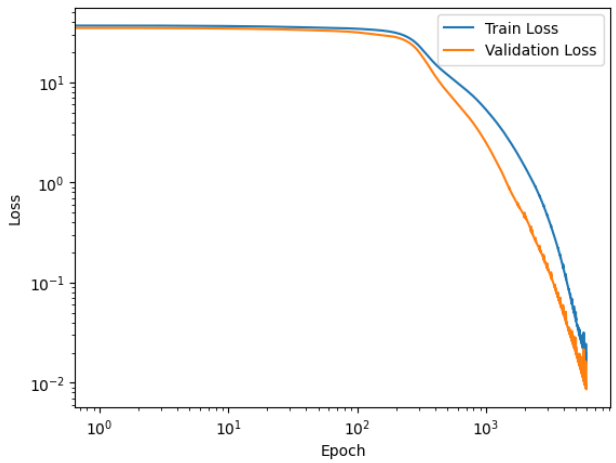
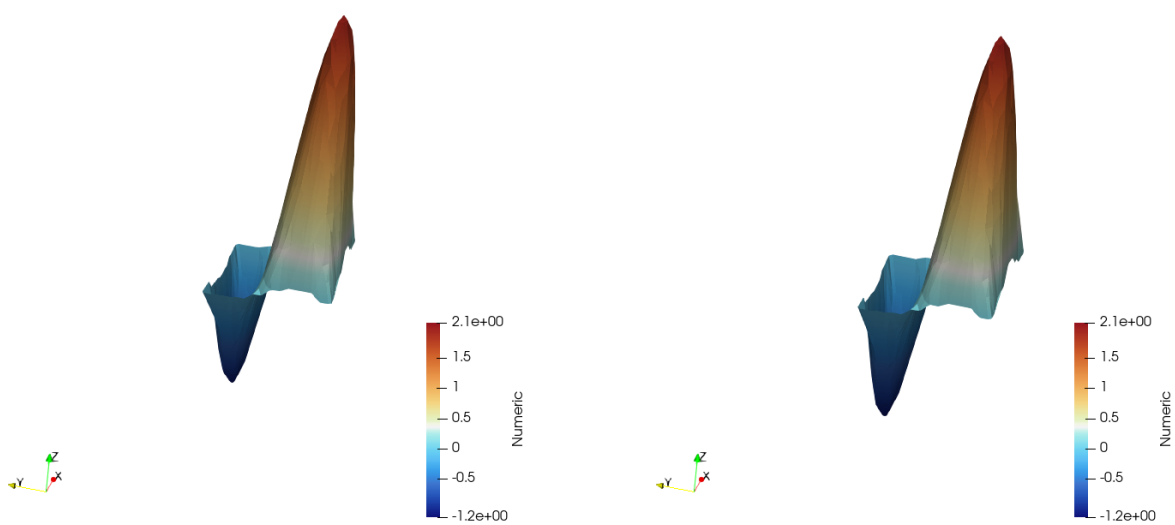


Figure 9: Training loss (Blue) and validation loss (Orange) of the PODnn network for Ex.2.



(a) High-fidelity solution.

(b) POD-NN solution.

Figure 10: Comparison between high-fidelity and POD-NN solutions for test parameters $\mu_0 = 0.455$ and $\mu_1 = 0.72$.

Table 10: Performance metrics for POD-NN (Experiment 2)

Method	Avg Abs Error	Avg Rel Error	Speed-Up
POD-NN	0.011	0.0034	35.7

Despite a slight loss of accuracy compared to standard POD, the POD-NN approach allows for a pretty good speed-up, since the solution is obtained in a non-intrusive and fully non-iterative way. However, based on the current stage of our experiments, it is evident that the complex network architecture used for the POD-NN acts as a limiting factor. It represents an overkill for this problem. We are confident that, by adopting a much lighter architecture, similar to the one redesigned for the PINN, a significantly higher speed-up could be achieved without compromising accuracy.

4.4 Comparison between methods for ex.2

Table 11 summarizes the accuracy and computational performance of the different methods applied in Experiment 2. As in the previous case, the High-Fidelity (HF) model acts as the reference in terms of accuracy.

The POD method maintains a low average error and good accuracy but offers only poor computational savings because of the expensive online phase caused by the iterative solver (for the non linearity) and by the reconstruction of almost all the matrix and vectors in the non affine case. The POD-NN variant, while slightly less accurate, achieves a significant speed-up, confirming its effectiveness as a fast surrogate without major compromises in precision.

The PINN approach, once again, exhibits the largest errors, though it benefits from faster inference. Interestingly, the simplified PINN architecture, thanks to structural adjustments, reaches a much higher speed-up (over 400×) while retaining roughly the same error levels. This underlines the strong impact that architectural simplifications can have on computational performance, although without solving the accuracy limitations of PINNs in this setting.

Overall, the trends from Experiment 1 are confirmed. POD-based techniques offer the best accuracy, while PINNs achieve remarkable speed. Among the tested strategies, POD-NN continues to stand out as the most balanced solution. Exploring simplified architectures for POD-NN, similar to those adopted for PINNs, could further enhance its performance, and represents a valuable direction for future experiments.

Table 11: Performance comparison of methods for Experiment 2

Method	Avg Abs Error	Avg Rel Error	Speed-Up
HF	0	0	1
POD	0.0094	0.0029	1.40
PINN	0.5190	0.0877	29.5
PINN (Simplified)	0.369	0.062	415.0
POD-NN	0.0110	0.0034	35.7

5 Conclusions

Comparing the results obtained with the proposed model order reduction methods, the most effective approach is POD-NN. This method achieves an excellent trade-off between accuracy and computational efficiency. In contrast, while POD delivers the highest accuracy, its computational cost is significantly higher especially in the context of non-linear PDEs like the one analyzed in this study.

PINNs also represent a valid alternative when the priority is speed rather than precision, but the crucial part is the network design. They are particularly attractive when a fast approximate solution is acceptable, even at the cost of increased error.

Additionally, a comparison between the two experiments highlights the impact of the forcing term on the performance of the reduction methods. In Section 4, the forcing term depends on the parameter $\boldsymbol{\mu}$ in

a non-affine manner. This breaks the affine structure assumed for the forcing term, resulting in a higher-dimensional POD basis (dimension 9), which is three times larger than in Section 3. Both the POD and POD-NN methods show a slight decline in performance in Section 4, with reduced efficiency and/or increased error compared to their results in Section 3. In contrast, the PINN approach, which is primarily sensitive to the complexity of the training data rather than affine decomposability, appears unaffected by the added complexity of the problem. It achieves comparable accuracy, while benefiting from a significantly higher speed-up, almost entirely given by a greater computational cost of the HF solution in this second case.

References

[1] C. Canuto. Notes on partial differential equations, July 2009.

[2] C. Canuto. Notes of the lecture course numerical methods for partial differential equations, January 2025.

[3] F. Vicini and M. Strazzullo. Course notes for model order reduction and machine learning. Lecture notes, 2025. Available via course website or internal distribution.

A FEM convergence orders

Using the well-known exact solution $u = 16xy(1-x)(1-y)$ the error of the HF solution varying the mesh size is computed and, if $u \in H^{k+1}(\Omega)$, it is equal to:

$$\|u - u_\delta\|_{L^2(\Omega)} \leq Ch^{k+1}|u|_{H^{k+1}(\Omega)}$$
$$\|u - u_\delta\|_{H^1(\Omega)} \leq Ch^{k}|u|_{H^{k+1}(\Omega)}$$

with h mesh size, C constant h-indipendent and k order of the FEM method with the Lagrangian basis in $\mathbb{P}_k$.

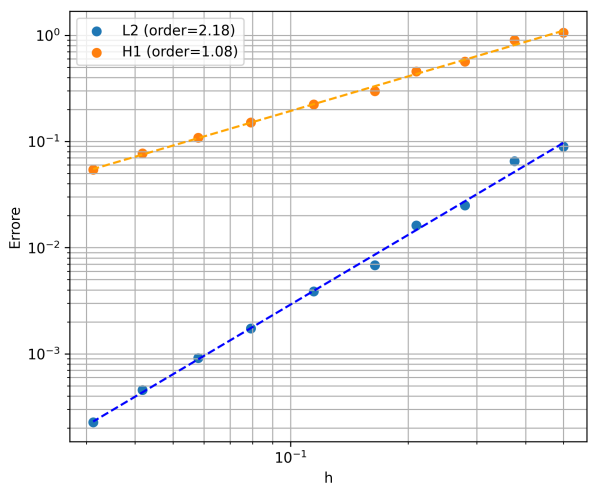


Figure 11: Convergence of order one FEM ($\mathbb{P}_1$) in log-log scale

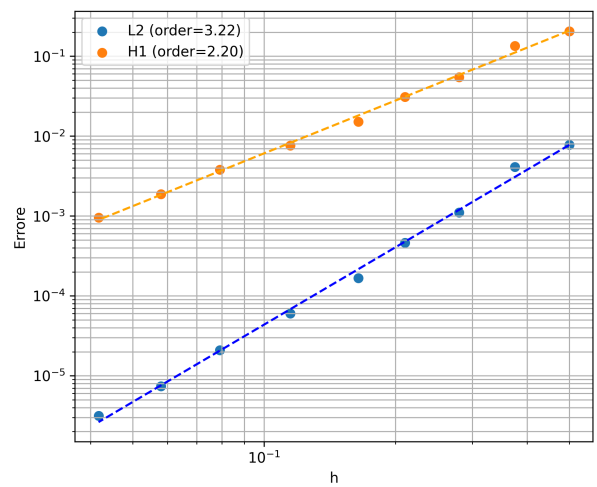


Figure 12: Convergence of order two FEM ($\mathbb{P}_2$) in log-log scale

B POD truncation error convergence

Here are proposed the plots of the sum of the eigenvalues varying N. Note that the error of POD is strictly related to this sum. In particular, the POD truncation error decreases as the number of r retained modes increases, and its decay is directly related to the spectrum of the correlation (snapshot) matrix, specifically the decay of its eigenvalues

$$\|u_\delta - u_N\|^2 = \sum_{i=r+1}^{N} \lambda_i, \qquad \text{and} \qquad \lambda_i \sim Ce^{-\alpha i}$$

where C and α are constants. Two different values of λ_i are proposed with reference to the scalar product used in order to construct the covariance matrix.

$$(u,v)_{L^2(\Omega)} = \int_\Omega uv \qquad \text{or} \qquad (u,v)_{H^1(\Omega)} = (u,v)_{L^2(\Omega)} + (\nabla u, \nabla v)_{L^2(\Omega)}$$

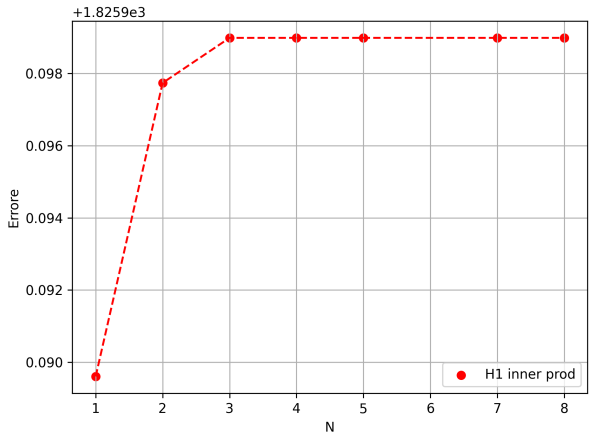


Figure 13: POD error in H^1 norm for example 1

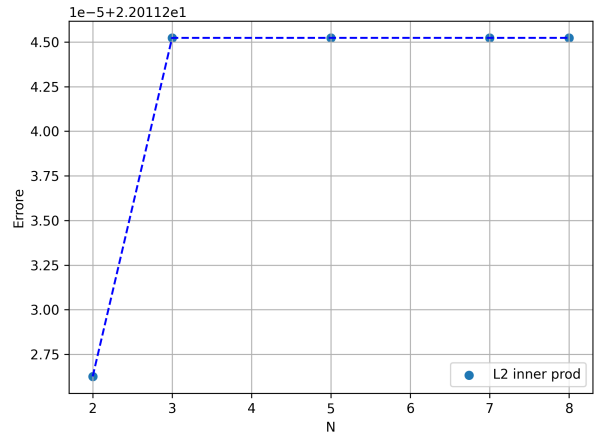


Figure 14: POD error in L^2 norm for example 1

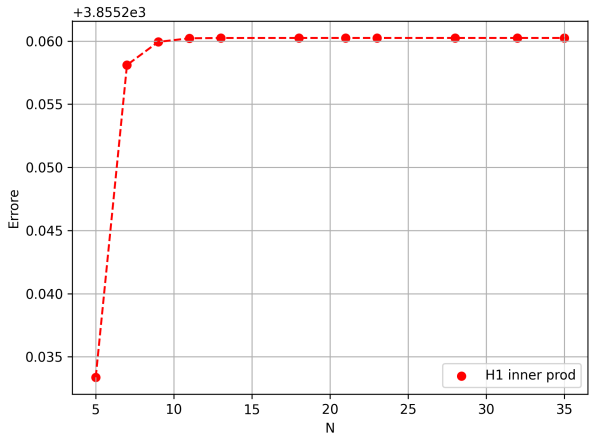


Figure 15: POD error in H^1 norm for example 2

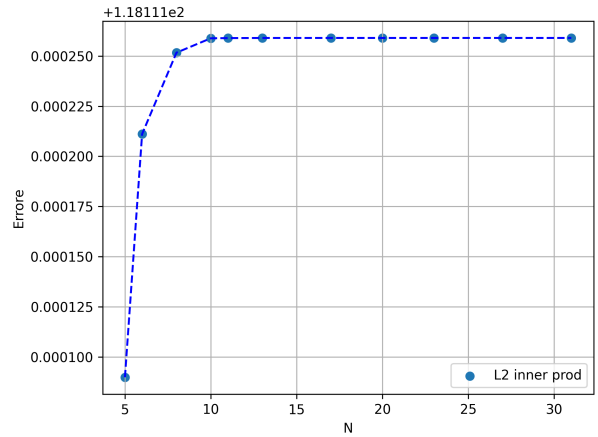


Figure 16: POD error in L^2 norm for example 2